Time Is the Baseline to Beat: Lead Time over a Timer as the Test for VLA Failure Monitors

Author Names Omitted for Anonymous Review. Paper-ID [add your ID here]

***Abstract*—On RoboCasa365, a failed vision-language-action (VLA) rollout is one that reaches its time limit. A runtime failure monitor therefore competes with a timer that alarms at a fixed fraction of that limit. Calibrated at a 5% false-positive rate (FPR), such a timer flags every one of 380 prospective failures, about 12 policy inferences before the timeout. Recall cannot separate a learned monitor from this timer; only lead time can. We state this as a testable hypothesis: at a matched calibrated FPR, alarming when *either* a policy-internal score or the timer crosses its threshold warns earlier than the timer and than any combination of time signals alone. We give a split-conformal calibration of this two-threshold rule with a finite-sample FPR guarantee. Exploratory evidence on 760 prospective rollouts of one VLA on 38 tasks favours time. Fusing the SAFE failure score with a single timer raises the mean warning lead from 0.090 to 0.207 of the time limit (paired gain 0.117, 95% CI 0.060–0.178), and noise fused with that timer reaches only 0.101. But a per-task timer, which alarms past the 95th percentile of each task's development success durations plus a conformally calibrated margin, reaches 0.195 at 0.045 FPR without reading the policy, and adding SAFE to it gains only 0.007 (CI -0.003 to 0.019). Stage 2 tests the hypothesis against this stronger time baseline on a preregistered fresh cohort and a second VLA, and asks whether earlier alarms raise task success within a fixed time budget when they trigger a reset and retry.**

I. Introduction

Runtime monitors for VLA policies read the policy's hidden state and predict eventual failure [8], or score how unusual that state is relative to successful rollouts [27, 19]. They are compared with each other, but not with what the robot knows for free: how long the attempt has been running.

In RoboCasa365, as in most simulated manipulation benchmarks, an episode ends at first success or at a time limit, and in our test cohort every one of 380 failed rollouts ran to its limit. A timer at 90% of the limit therefore flags all of them at 4.7% FPR. Against it, a monitor can add only one thing: an earlier alarm at the same false-alarm rate, which leaves time to stop, reset or ask for help. We make this lead time the test, with three contributions.

1) **A testable hypothesis and endpoint.** We define the warning lead of an alarm as the fraction of the time limit left when it fires (zero for a missed failure) and state four hypotheses with decision rules fixed before Stage 2 data are collected (Sec. IV).
2) **Calibrated OR-fusion with a guarantee.** We alarm when either a policy-internal score or the timer crosses its threshold, and calibrate both thresholds along a nested path with a split-conformal rule that bounds the FPR on a new successful rollout by α in finite samples (Proposition 1).
3) **Preliminary evidence under a strict protocol.** With thresholds frozen on a separate calibration cohort and evaluated once on a disjoint prospective cohort, SAFE fused with a single timer more than doubles its mean lead, but a per-task timer that never reads the policy matches it, and SAFE adds only 0.007 on top (Sec. VI).

The preliminary data point to no useful effect. A negative confirmatory result would still be informative: on timeout-defined benchmarks, policy internals would add little to duration statistics, and monitors should be evaluated on failures that end early.

II. Related Work

Failure detection from policy internals. SAFE learns a failure score from a VLA's last-layer features and thresholds it with a functional conformal band [8]; we reuse its implementation. Follow-ups change the training data [18], labels [17, 14], features [28, 31] or policy [22]. None reports an elapsed-time detector. SAFE removes the duration shortcut by truncating rollouts to equal length [8], which a deployed monitor cannot do. Duration is a shortcut feature [7], but its causal form is legitimate evidence online, so we make it the baseline.

Action-space and success-only detectors. STAC [1] and VLA-FAIL [20] compare overlapping action chunks, and ActProbe [9] feeds chunk consistency and magnitude to a task-conditioned LSTM, beating internal-feature baselines on RoboCasa. Our policy executes each chunk in full, so overlap signals are undefined; chunk magnitude and a second flow sample are Stage 2 baselines. A task-conditioned recurrent detector can also learn how long each task takes, which our timers make explicit. FAIL-Detect [27] and FIPER [19] score inputs, outputs or embeddings against successful rollouts only. FIPER alarms when two conformal scores *both* exceed their thresholds, which suppresses false alarms; we alarm when *either* crosses, because the policy score and the timer catch failures at different times, and calibrate the pair jointly.

World-model verifiers and VLM monitors. Check-VLA [12] is the closest work on our benchmark. It checks each action chunk against an action-conditioned world model on RoboCasa365, calibrates a conformal threshold to a 5% episode-level rate of unnecessary interventions, and repairs the chunk with the same VLA. Foresight [24] uses world-model latents. Neither compares against elapsed time, and either risk

could be fused with a timer as we fuse SAFE. LLM and VLM monitors [1, 13, 21, 30, 6] are slower and complementary.

Time, calibration and intervention. VLA confidence drifts over a task [32]. Our time-risk baseline is a per-task survival-style estimate [10, 5], and lead time at a fixed false-alarm budget is the accuracy–earliness trade-off of early time-series classification [26]. Split-conformal thresholds give finite-sample false-positive control under exchangeability [3]; we extend the construction to a two-signal OR rule (Sec. V-C). RESCUE [2] finds that good detection scores do not guarantee lower intervention cost, and CheckVLA and FLARE [29] couple detection to repair. Hence our closed-loop Stage 2 test: an earlier alarm matters only if acting on it raises success.

III. Problem Setting

A monitor watches a rollout of a known task k and decides at each policy inference $t = 1, 2, \ldots$ (one per 16-step action chunk) whether to alarm, using only the policy features so far, t, the task and its time limit H_k (in inferences). The first crossing is the alarm T. FPR is the fraction of successful rollouts with any alarm. For a failure, which ends at H_k, the *warning lead* is $\ell = (H_k - T)/H_k$, or 0 if missed; the mean lead $\bar{\ell}$ combines recall and earliness. One inference is 16 control steps, or 0.8 s at 20 Hz[CHECK: confirm the control rate used in the evaluation]. Detectors are compared at operating points fixed on a calibration cohort under FPR $\leq \alpha = 0.05$. Confidence intervals come from a paired bootstrap that resamples tasks, then rollouts within each task (2,000 replicates).

IV. Hypotheses

We test four hypotheses. H1 and H2 are evaluated on a fresh confirmatory cohort with the decision rules below, fixed before that cohort is opened; H3 and H4 are the Stage 2 extensions (Sec. VII).

H1 (lead time, primary). At thresholds calibrated for FPR ≤ 0.05, OR-fusing a policy-internal score with the per-task timer (Sec. V), the strongest time-only monitor we know, increases the mean warning lead over that timer alone. *Supported* if the paired bootstrap 95% CI of $\Delta\bar{\ell}$ lies above zero, the point gain is at least 0.05 of the time limit, and the fused test FPR is at most 0.075 with a Wilson 95% upper bound at most 0.10. As a secondary check, the fused detector must also beat a timer whose fraction is tuned *on the test cohort* to the fused detector's test FPR, which favours the timer.

H2 (the gain needs policy evidence). Fusions that contain no policy-internal evidence (Gaussian noise, or SAFE scores resampled within task, each fused with the per-task timer) gain less than 0.05 of the time limit over the per-task timer.

H3 (generality). H1 holds for a second VLA with a different architecture.

H4 (utility). When an alarm triggers a reset and retry, the OR-fused monitor yields a higher task success rate than the timer within the same per-task time budget.

Only H1 and H2 are confirmatory; every other interval in this paper is descriptive. On the opened data the H1 gain is 0.007, so we expect H1 to fail; that outcome would show that, when failure means timeout, a policy's hidden state adds no usable warning over task duration.

V. Method

A. Signals

Policy score. At each inference we take the final-layer action-token hidden state of the policy's action head (1,024 dimensions). The SAFE MLP [8] maps it to a failure score $u_t^{(j)}$ for each of three training seeds j. With per-task location $m_k^{(j)}$ and scale $\sigma_k^{(j)}$ estimated on development rollouts, the policy score is

$$s_t = \frac{1}{3}\sum_{j=1}^{3} \frac{\max_{t' \leq t} u_{t'}^{(j)} - m_k^{(j)}}{\sigma_k^{(j)}}. \tag{1}$$

The running maximum makes s_t monotone and does not change when a threshold is first crossed. Success-only alternatives (Mahalanobis [11], kNN [23], PCA-KMeans and RND [4]) are fitted on successful development rollouts and aggregated the same way.

Timers. The global timer alarms once $t \geq c\,H_k$ for a fraction $c \in (0, 1]$ shared by all tasks; it uses only the task's time limit. The per-task timer uses $c_k = \min(1, \rho_k + \delta)$, where ρ_k is the 95th percentile of task k's development success durations as fractions of H_k and the shared margin δ is set by Proposition 1 on calibration successes. It needs successful rollouts of each task, but no failures and no policy features. We fixed the 95th percentile after auditing the opened cohort, so we report 0.75, 0.90 and 1.0 (the longest success) as sensitivity.

Time risk (baseline). The survival-style alternative estimates $r_t = \Pr(\text{failure} \mid k, \text{still running at } t)$ from development rollouts: with n_t of them still running at t and f_t of those failing, it fits an increasing isotonic regression in t/H_k to $(f_t + 0.5)/(n_t + 1)$, weighted by n_t. Because every failure runs to H_k, r_t is a task-specific timer; its smoothing keeps it below a high threshold in tasks with few development failures, which is why it misses failures the plain timer catches.

B. OR-fusion

The fused monitor alarms at

$$T = \min\{\, t : s_t > \tau_s \text{ or } t \geq c\,H_k \,\}. \tag{2}$$

Neither signal is restricted in time: the policy score may alarm early, and the timer guarantees an alarm on every failure, at $c\,H_k$ at the latest. The lead gain over the timer comes only from failures on which the policy score crosses first.

C. Calibration with a finite-sample FPR guarantee

A successful rollout i of length L_i (in inferences) raises a false alarm under (τ_s, c) if and only if $M_i = \max_{t \leq L_i} s_t > \tau_s$ or $\rho_i = L_i/H_{k_i} \geq c$, so its false alarm depends only on the pair (M_i, ρ_i). Fix in advance a path $\gamma(\lambda) = (\tau_s(\lambda), c(\lambda))$, $\lambda \in [0, 1]$, with both coordinates continuous and nondecreasing, so

that a larger λ is more conservative and $\gamma(1)$ never alarms on a success. For each calibration success define

$$V_i = \min\{\lambda : M_i \le \tau_s(\lambda) \text{ and } \rho_i < c(\lambda)\}, \qquad (3)$$

the least conservative point on the path at which rollout i stays silent. Because the path is monotone, rollout i is silent at λ exactly when $\lambda \ge V_i$.[1]

Proposition 1: Let n calibration successes and a new successful rollout be exchangeable, and let the path γ not depend on the calibration successes. Set $\hat{\lambda} = V_{(\lceil (n+1)(1-\alpha) \rceil)}$, the $\lceil (n+1)(1-\alpha) \rceil$-th smallest V_i (or $\hat{\lambda} = 1$ if that rank exceeds n). Then the OR-fused monitor at $\gamma(\hat{\lambda})$ raises a false alarm on the new success with probability at most α.

Proof: The new rollout alarms at $\hat{\lambda}$ if and only if $V_{n+1} > \hat{\lambda} = V_{(r)}$ with $r = \lceil (n+1)(1-\alpha) \rceil$. By exchangeability, the rank of V_{n+1} among $V_1, \dots, V_{n+1}$ is uniform on $\{1, \dots, n+1\}$ (ties broken at random), so $\Pr(V_{n+1} > V_{(r)}) \le 1 - r/(n+1) \le \alpha$; this is the split-conformal quantile lemma [3] applied to the scalar V. ∎

The path may use any data except calibration successes, such as development rollouts or calibration *failures*. The guarantee is marginal over tasks; a per-task version needs 19 calibration successes per task. The deployed FPR is Beta-distributed given calibration: it exceeds 0.075 with probability 0.065 for $n = 114$, but 0.021 for $n = 380$.

The preliminary results below use the simpler joint search of the original protocol: among all pairs of observed calibration maxima with calibration FPR $\le \alpha$, keep the pair with the largest mean lead (highest recall for the time-risk fusions). Selecting the best of many non-nested pairs on the same 114 successes has no finite-sample guarantee and is optimistic, which matches the test FPRs of 0.058–0.068 we observe for two-threshold detectors against 0.044 on calibration. The per-task timer rows already use Proposition 1, with the path $\delta = \lambda$, $\tau_s = u + s\lambda$ and (u, s) chosen on calibration failures; Stage 2 uses it for every detector.

VI. Preliminary Evidence

Setup. We use the released Xiaomi-Robotics-1 RoboCasa365 checkpoint [25], which executes each 16-step action chunk in full, on the 38 official RoboCasa365 tasks [15, 16] with published success between 5 and 45 of 50 and enough failures (10 atomic, 28 composite; failures last 29–272 inferences). Disjoint cohorts serve one purpose each: development (1,824 rollouts) fits SAFE and the time risk; calibration (3 successes and 3 failures per task; 454 rollouts including those past the quota) sets thresholds; the prospective test (10 per class per task) is evaluated once; a confirmatory cohort of the same size is reserved for Stage 2. SAFE, its normalization and the time risk were frozen before the test was collected; the timers and fused detectors were added later, with thresholds from calibration only, so the results below are exploratory.

[1]The strict $\rho_i < c(\lambda)$ makes the set right-open; taking c on a finite grid, as the timer's resolution of one inference does anyway, makes the minimum attained.

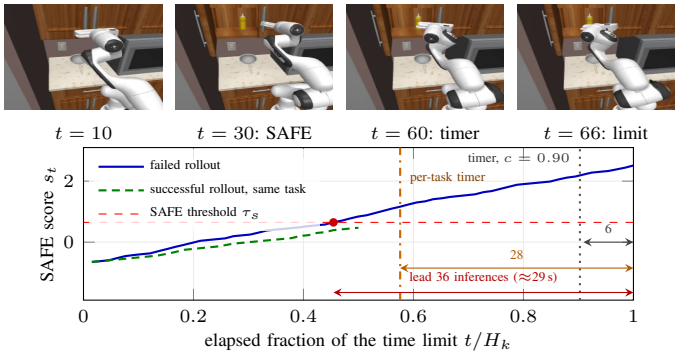


Fig. 1. One failed *OpenCabinet* rollout from the prospective test (chosen to illustrate; the median SAFE-first lead is smaller). The SAFE score crosses its calibrated threshold at $t = 30$ of $H_k = 66$ inferences, 36 inferences before the time limit; the timer alarms at $t = 60$, and the per-task timer, which never reads the policy, at $t = 38$. Dashed: the longest of the 9 (out of 10) successful test rollouts of this task that stay below the threshold; it ends at $t = 33$.

TABLE I

Prospective test (380 successes, 380 failures), thresholds from calibration at FPR ≤ 0.05. $\bar{\ell}$ = mean warning lead as a fraction of the time limit (miss = 0); Med = median lead in inferences over detected failures; @.75 = failures alarmed by $0.75\,H_k$. *Mean over 20 draws (range of $\bar{\ell}$ 0.088–0.134). †95th percentile of development success durations, margin by Proposition 1 on the 114 calibration successes.

Detector	Recall	FPR	$\bar{\ell}$	Med	@.75
Timer ($c = 0.903$)	1.000	0.047	0.090	11.5	0.000
Time risk	0.632	0.047	0.120	16.5	0.211
Time risk + timer	1.000	0.050	0.125	11.0	0.105
Noise + timer*	1.000	0.066	0.101	–	–
SAFE only	0.566	0.039	0.190	23.0	0.421
SAFE + time risk	0.955	0.058	0.184	12.0	0.311
Per-task timer†	1.000	0.045	0.195	13.5	0.342
SAFE + per-task timer†	1.000	0.042	0.202	14.0	0.389
SAFE + timer	1.000	0.058	0.207	10.0	0.421

The timer is the baseline to beat. The timer ($c = 0.903$) alarms on all 380 test failures at 0.047 FPR with a fixed lead of 0.090 (Table I). The time-risk curve misses 140 failures, all in 14 tasks whose smoothed curve never reaches its threshold, so recall gains measured against it (to 0.90–0.96 for SAFE and four success-only scores) vanish here.

SAFE adds lead over a single timer. SAFE + timer raises the mean lead to 0.207 (paired gain 0.117, CI 0.060–0.178; 20.2 against 11.7 inferences) and flags 42% of failures by $0.75\,H_k$ (Figs. 1 and 2), at 0.058 test FPR. Noise fused with the timer reaches 0.101, and the time risk with the timer 0.125. Among rollouts still running at 10, 25, 50 and 75% of the limit, SAFE's within-task ROC-AUC is 0.55–0.60, against 0.41–0.51 for noise. Bootstrapping calibration as well, with thresholds re-selected, keeps the gain positive (0.125, CI 0.031–0.246) but lets the fused test FPR reach 0.16, so threshold uncertainty dominates. On test-cohort frontiers, OR and AND fusion reach the same lead at 5% FPR (0.208 and 0.211); OR keeps the timer's guaranteed alarm.

A per-task timer removes the gain (H1, exploratory). The per-task timer reaches $\bar{\ell} = 0.195$ at 0.045 FPR without

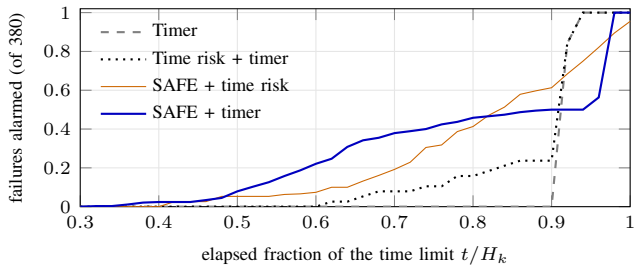


Fig. 2. Fraction of the 380 test failures alarmed by each elapsed fraction of the time limit. SAFE + timer catches half of the failures early and the rest at its timer fallback ($c = 0.96$).

reading the policy. SAFE + timer leads it by an insignificant 0.012 (CI -0.044 to 0.063), and by -0.003 against a per-task timer tuned on test to the same FPR. SAFE + per-task timer reaches 0.202, a gain of 0.007 (CI -0.003 to 0.019), far below the 0.05 margin of H1. The two effects cancel across tasks: in the 14 tasks where the time risk never fires, SAFE + timer's mean lead is 0.405 against 0.221 for the per-task timer; in the other 24 it is 0.092 against 0.180. The percentile matters: with the longest development success ($q = 1$) the per-task timer reaches only 0.125 and SAFE adds 0.057 (CI -0.003 to 0.119); with $q = 0.90$, 0.169 and 0.028 (CI -0.012 to 0.067).

No early warning yet. No detector flags more than 8% of failures by half of the limit; whether a late warning is usable is H4.

VII. Stage 2 Plan

E1: Confirmatory test of H1 and H2. A fresh cohort of 760 rollouts (10 per class per task, new environment seeds) is being collected and has not been opened. Before opening it, we amend its preregistration with the per-task timer (95th percentile of development successes, margin by Proposition 1) as the primary comparator and $\Delta\bar{\ell}$ as the primary endpoint, with the H1 and H2 decision rules of Sec. IV. The earlier recall endpoint against the time risk becomes secondary. We enlarge calibration to 10 successes per task ($n = 380$), apply Proposition 1 with a path chosen on development data, and bootstrap the calibration cohort as well as the test cohort, re-selecting thresholds in each replicate, so the intervals include threshold uncertainty.[PENDING: merge the amendment (PR #3) and name the calibration seed range before opening the confirmatory cohort.]

E2: Second policy (H3). We repeat the timer, time-risk, SAFE and fused rows with π_0 or RLDX-1 on the same 38 tasks, reusing saved rollouts where hidden states were recorded. A policy with overlapping chunks also lets us run chunk-consistency monitors [1, 9].

E3: Closed-loop reset and retry (H4). On 8 to 10 tasks with mid-range success, each episode gets a budget of $2H_k$ inferences. Under each monitor (none, timer, SAFE + timer), an alarm ends the attempt, resets the scene to its initial state and starts a new attempt while budget remains; without a monitor, the retry starts at the timeout. We compare success within the budget, paired by environment seed, and count successes aborted by false alarms. The gain is bounded by the failure rate times the retry success rate times the fraction of failures whose extra lead leaves room for a retry; a null result would bound the value of any such monitor on this benchmark.

E4: Stronger baselines and checks. (i) Action-only monitors that need no hidden state: chunk magnitude and the disagreement of a second flow sample. (ii) Richer time baselines: a stall detector on end-effector motion and a discrete-time hazard model on simple observables, so that "time" means the best signal available without the policy's internals. (iii) OR- and AND-fusion compared on full recall–FPR curves rather than single points. (iv) SAFE faithfulness checks and precision at deployment prevalence. (v) An unseen-task split, holding tasks out of SAFE training and normalization; the timer needs only H_k and transfers as is.

E5: Failures that end early. Every failure here ends at the limit. We will label, on a subset, when a failure became unrecoverable (a dropped or displaced object), and report lead relative to that moment, since these are the failures a deployed robot most needs to catch early.

VIII. Limitations

The preliminary results come from one policy in simulation, on a cohort opened before the fused detectors and the timer were defined; only E1 can make them confirmatory. The two-threshold detectors exceed the FPR budget on test, which Proposition 1 and a larger calibration cohort address. The timer, the time risk and the per-task normalization assume that the task and its time limit are known, and the policy score needs white-box access to the policy and development rollouts of each task; on our data, that access bought 0.007 of lead over the per-task timer. Test cohorts are balanced by quota, so recall and FPR are per class rather than at deployment prevalence.

IX. Conclusion

When failure means timeout, a calibrated timer catches every failure, and the question for a learned monitor becomes how much earlier it can warn at the same false-alarm rate. On one VLA and 38 tasks, fusing a hidden-state failure score with a single timer more than doubled the mean warning lead, but a per-task timer built from development success durations matched it without reading the policy, and the hidden state added 0.007 on top of it. Stage 2 tests this against the per-task timer on a fresh cohort and a second policy, with a finite-sample FPR guarantee, and asks the question that decides its value: whether the extra warning lets a robot recover more often within the same time.

References

[1] Christopher Agia, Rohan Sinha, Jingyun Yang, Zi-ang Cao, Rika Antonova, Marco Pavone, and Jeannette Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Conference on Robot Learning (CoRL)*, volume 270 of *PMLR*, 2024.

[2] David Ahmedt-Aristizabal, Abdelwahed Khamis, Mohammad Ali Armin, and Lars Petersson. RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors, 2026. arXiv:2610.13516.

[3] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[4] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.

[5] David R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B*, 34(2):187–220, 1972.

[6] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2 (11):665–673, 2020.

[8] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. SAFE: Multitask failure detection for vision-language-action models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[9] Bingjia Huang, Xiangyu Li, Xiang Wang, Liang Mi, Zixu Hao, Weijun Wang, Hao Wu, Kun Li, Yunxin Liu, and Ting Cao. ActProbe: Action-space probe for early failure detection of generative robot policies, 2026. arXiv:2606.08508.

[10] Edward L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.

[11] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[12] Yushan Liu, Peibo Sun, Xintao Chao, Zhenyang Yang, Yifan Xie, Lingfeng Zhang, Shoujie Li, Chenyu Tang, Fang Chen, Xiao-Ping Zhang, and Wenbo Ding. CheckVLA: Execution-time verification with action-conditioned world model for long-horizon mobile manipulation, 2026. arXiv:2607.26789.

[13] Zeyi Liu, Arpit Bahety, and Shuran Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, 2023.

[14] Jie Ma, Zongxi Liu, and Yi Zhu. FailureSpot: Label-efficient timestamp-level failure detection for vision-language-action models, 2026. arXiv:2609.04277.

[15] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. RoboCasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.

[16] Soroush Nasiriany, Sepehr Nasiriany, Abhiram Maddukuri, and Yuke Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations (ICLR)*, 2026.

[17] Seongheon Park, Wendi Li, Changdae Oh, Min-Hsuan Yeh, Zsolt Kira, Michael Hagenow, and Sharon Li. Hide-and-seek in trajectories: Discovering failure signals for VLA runtime monitoring. In *ICML Workshop on Failure Modes in Agentic AI*, 2026. arXiv:2605.30834.

[18] Harshitha Rajaprakash, Aditeya Prajapati, Rong Xue, Abrar Anwar, and Jesse Thomason. SAFECAST: Robust failure detection for VLA policies with contrast-set training and calibration, 2026. arXiv:2608.04246.

[19] Ralf Römer, Adrian Kobras, Luca Worbis, and Angela P. Schoellig. Failure prediction at runtime for generative robot policies. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[20] Florian Seligmann, Emiliyan Gospodinov, Enes Ulas Dincer, and Gerhard Neumann. VLA-FAIL: Efficient task failure detection for finetuned vision-language-action models, 2026. arXiv:2606.21386.

[21] Rohan Sinha, Amine Elhafsi, Christopher Agia, Matthew Foutter, Edward Schmerling, and Marco Pavone. Real-time anomaly detection and reactive planning with large language models. In *Robotics: Science and Systems (RSS)*, 2024. doi: 10.15607/RSS.2024.XX.114.

[22] Michael Soddy, Karthikeyan Ganesan, Roman Stemp, Ved Deo, and Nicholas Babey. Predictable before action, resistant to correction: Failure signals in vision-language-action models. In *NeurIPS Workshop on From Pretrained Representations to Acting Agents (PTA)*, 2026.

[23] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *International Conference on Machine Learning (ICML)*, 2022.

[24] Boyang Wang, Haoran Zhang, Mengdi Wang, Odest Chadwicke Jenkins, Xuhui Kang, Yen-Ling Kuo, Yifu Lu, and Zezhou Cheng. Foresight: Failure detection for long-horizon robotic manipulation with action-conditioned world model latents, 2026. arXiv:2606.23085.

[25] Xiaomi Robotics Team. Xiaomi-Robotics-1: Scaling vision-language-action models with over 100K hours of real-world trajectories, 2026. arXiv:2607.15330.

[26] Zhengzheng Xing, Jian Pei, and Philip S. Yu. Early classification on time series. *Knowledge and Information Systems*, 31(1):105–127, 2012. doi: 10.1007/s10115-011-0400-x.

[27] Chen Xu, Tony Khuong Nguyen, Emma Dixon, Christopher Rodriguez, Patrick Miller, Robert Lee, Paarth Shah, Rares Ambrus, Haruki Nishimura, and Masha Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025.

[28] Jinghan Yang, Yunchao Zhang, Wang Yuan, Haolun Wan, Jiaming Zhang, Zhengyang Hu, and Yanchao Yang. Tri-Info: Generalizable, interpretable failure prediction for VLA models via information theory, 2026. arXiv:2606.19998.

[29] Ganlong Zhao, Zijia Tang, Xingping Chen, Zhanghui Kuang, Ye Tian, and Guanbin Li. FLARE: A failure-aware framework for autonomous correction and recovery in visual-language robotic manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22391–22401, 2026.

[30] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6919–6929, 2025.

[31] Kaiwen Zhu, Dongfang Liu, and Liangkai Liu. VLA-Scope: Shift-aware failure prediction for vision-language-action models, 2026. arXiv:2609.21246.

[32] Tom Zollo and Richard Zemel. Confidence calibration in vision-language-action models. In *ICML Workshop on Statistical Frameworks for Uncertainty in Agentic Systems*, 2026. arXiv:2507.17383.